87

Distributed Computation and Networks
A Performance Perspective

Assignment 2:
Performance of a Network Router

Alessandro Versini *
(av2013)

February 21, 2017

*Declaration: We confirm that this submission is our own work. In it, we give references and citations whenever we refer to or use the published, or unpublished, work of others. We are aware that this course is bound by penalties as set out in the College examination offenses policy.

Contents

1 Introduction **3**

2 Theoretical Background **3**
2.1 The State Variable: Number of Packets in the Queue 4
2.2 Router Utilisation 5
2.3 Number of Packets in the Queue 6
2.4 Waiting Time 7

3 Discrete Event Simulation Program **7**
3.1 Random Number Generator 7
3.2 Implementation 11
3.3 Experimental Results 13

A Code Listing **17**
A.1 Main Function 17
A.2 Simulator 21
A.3 Random Number Generator 23

1 Introduction

The router is a fundamental component in a packed switching network. First introduced in the context of the ARPANET in 1969 [1], it is still today the device that allows billions of packets to be delivered over the Internet. More specifically, the router is responsible for packets forwarding from one node of the network to the other. It has two (or more) I/O lines connecting it to adjacent nodes: when a packet arrives at the router, its header, which contains the address of its ultimate destination, is read and the packet is sent to the next node accordingly. The correct adjacent node which to forward the packet to is usually decided by means of a table lookup.

In this Assignment we would like to study the performance of a router mainly as a function of the packet arrival rate and the average servicing time (i.e. the time the router takes to process a packet). After a brief discussion on the theoretical basis we will present the design of a discrete event simulation program which assumes Poisson packet arrivals and exponentially distributed servicing time. The simulator will be based on our own random number generator.

As a final remark it should be noted that in this Assignment we will be concerned with the microscopic analysis of the network performance, that is not at system level but rather at component level. Macroscopic network analysis does not usually treat the stream of information as a series of discrete packets transmissions but rather as a continuous flow of *fluid* in a *pipe* of limited capacity.

2 Theoretical Background

Our model of the router consists of two main blocks:

- A queue which is filled with Poisson distributed input packets;
- A processing unit that takes an exponentially distributed time t to serve one item in the queue;

Furthermore we assume that packets are processed on a first come first serve basis, that is the queue has a FIFO structure.

As far as the performance measure is concerned, we are interested in three parameters:

- The router utilisation, defined as the fraction of time that the processing unit is busy;
- The waiting time of a packet in the queue;
- The number of packets in the FIFO queue;

As we will se, while the waiting time and the queue length are linked by Little's theorem, a trade off exists between the router utilisation and the remaining two performance measures. This is intuitive as to increase the router utilisation one must increase the input packet rate which in turn increases the probability of an higher number of packets in the queue and consequently an higher waiting time for a fresh packet to be served.

We will now formalise these arguments and find theoretical expressions of the three performance measures in function of the input packet rate and average servicing time. We are required to study the first order statistics; however, for completeness and interest we will extend our analysis to include the second order statistics as well.

2.1 The State Variable: Number of Packets in the Queue

The number of packets in the queue is a random variable and from its statistics one can determine mathematical expressions for all the three performance measures discussed in previous paragraphs. To determine such statistics we start by defining the following probability:

$$P_n(t) = Pr\left[N(t) = n\right] \tag{1}$$

In words, this is the probability of the number of queued packets at time t ($N(t)$) being equal to n.

Elaborating from Equation 1, we can write:

$$\begin{aligned} P_n(t+\Delta t) =& (1-\mu\Delta t + o(\Delta t))(\lambda\Delta t + o(\Delta t)) \cdot P_{n-1}(t) + \\ & (1-\mu\Delta + o(\Delta t))(1-\lambda\Delta t + o(\Delta t)) \cdot P_n(t) + \\ & (\mu\Delta + o(\Delta t))(1-\lambda\Delta t + o(\Delta t)) \cdot P_{n+1}(t) \\ P_0(t+\Delta t) =& (1-\lambda\Delta t + o(\Delta t)) \cdot P_0(t) + \\ & (\mu\Delta t + o(\Delta t)) \cdot P_1(t) \end{aligned} \tag{2}$$

Taking the limit for $\Delta t \to 0$, (2) becomes a system of differential equations in t which cannot be solved in a closed form:

$$\begin{aligned} \frac{d}{dt}P_n &= \lambda P_{n-1}(t) + (\mu - \lambda)\, P_n + \mu P_{n-1} \\ \frac{d}{dt}P_0 &= -\lambda P_0(t) + \mu P_1(t) \end{aligned} \tag{3}$$

Nevertheless, if the transient response is neglected, a closed form steady state ($t \to \infty$)[1] solution to the system of differential equations can be derived using

[1] The process becomes ergodic, i.e. its statistics no longer depend on time.

the moment generating function, yielding:

$$P_n = P_0 \cdot \left(\frac{\lambda}{\mu}\right)^n \tag{4}$$

where P_0 can be determined from the condition:

$$\sum_{n=0}^{\infty} P_n = 1 \tag{5}$$

Solving Equation 5 we get:

$$P_0 = 1 - \frac{\lambda}{\mu} \tag{6}$$

which substituted back into Equation 4 yields a final expression for P_n:

$$P_n = \left(1 - \frac{\lambda}{\mu}\right) \cdot \left(\frac{\lambda}{\mu}\right)^n \tag{7}$$

It is important to stress that, since all P_n has to be positive and smaller than 1 $\forall n$, this result holds for $0 < \frac{\lambda}{\mu} < 1$.

From Equation 7 a few important results follow, including closed form expressions for the statistics of the router utilisation, the waiting time and the queue length. We shall derive such expressions in the next paragraphs.

2.2 Router Utilisation

Let us regard the router as *utilised* if its processing unit is active and *unutilised* if it is idle. Furthermore let us observe that the router can be processing packets only if the queue is non empty. It follows that the router utilisation is a boolean random variable that can be defined as:

$$U = \begin{cases} 1 & \text{, if the queue is non-empy} \\ 0 & \text{, otherwise} \end{cases} \tag{8}$$

From previous analysis, the probability of the queue being empty is:

$$P_0 = 1 - \frac{\lambda}{\mu} \tag{9}$$

and therefore the probability of the queue being non-empty is:

$$P_{N \neq 0} = 1 - P_0 = 1 - \left(1 - \frac{\lambda}{\mu}\right) = \frac{\lambda}{\mu} \tag{10}$$

It follows that the expectation of U is:

$$E[U] = 0 \cdot P_0 + 1 \cdot P_{N \neq 0} = \frac{\lambda}{\mu} \tag{11}$$

With a similar argument we can also calculate second order statistics:

$$Var[U] = E[U^2] - E^2[U] = \frac{\lambda}{\mu} - \left(\frac{\lambda}{\mu}\right)^2 \tag{12}$$

2.3 Number of Packets in the Queue

In statistical terms, the average number of packets in the queue is expressed as:

$$E[N] = \sum_{n=0}^{\infty} n \cdot P_n \tag{13}$$

which is nothing else than the expected value of the router state variable assuming the queue is allowed to grow to infinity.

Solving Equation 13 yields:

$$E[N] = \sum_{n=0}^{\infty} n \cdot P_n = \frac{\frac{\lambda}{\mu}}{1 - \frac{\lambda}{\mu}} \tag{14}$$

Out of interest, we can also calculate the theoretical variance of N as $Var[N] = E[N^2] - E^2[N]$. While we can obtain $E^2[N]$ by squaring the result of Equation 14, $E[N^2]$ has to be calculated as:

$$E[N^2] = \sum_{n=0}^{\infty} n^2 P_n = \left(1 - \frac{\lambda}{\mu}\right) \sum_{n=0}^{\infty} n^2 \left(\frac{\lambda}{\mu}\right)^n \tag{15}$$

Noting that (15) is equal to:

$$\left(1 - \frac{\lambda}{\mu}\right) \cdot \left(\frac{d^2}{dx^2} \sum_{n=0}^{\infty} x^{n+2}\Big|_{x=\frac{\lambda}{\mu}} - 3\sum_{n=0}^{\infty} n x^n - 2\sum_{n=0}^{\infty} x^n\right) \tag{16}$$

we can finally get an expression for $E[N^2]$:

$$E[N^2] = \frac{\left(\frac{\lambda}{\mu}\right)^2 + \frac{\lambda}{\mu}}{\left(1 - \frac{\lambda}{\mu}\right)^2} \tag{17}$$

Combining (14) and (17) yields the expression for $Var[N]$:

$$Var[N] = E[N^2] - E^2[N] = \frac{\frac{\lambda}{\mu}}{\left(1 - \frac{\lambda}{\mu}\right)^2} \tag{18}$$

2.4 Waiting Time

Going from the expected queue length to the average waiting time is straightforward. Little's Theorem states that:

$$E[N] = \lambda \cdot E[T_w] \tag{19}$$

where $E[N]$ is the average number of packets in the system, λ is the packet arrival rate and $E[T_w]$ is the expected waiting time of each packet. It follows that:

$$E[T_w] = \frac{E[N]}{\lambda} = \frac{\frac{1}{\mu}}{1 - \frac{\lambda}{\mu}} \tag{20}$$

Similarly the variance $Var[T_w]$ can be written as:

$$Var[T_w] = Var\left[\frac{N}{\lambda}\right] \tag{21}$$

which by properties of the variance becomes:

$$Var\left[\frac{N}{\lambda}\right] = \frac{1}{\lambda^2} Var[N] = \frac{\frac{1}{\mu\lambda}}{\left(1 - \frac{\lambda}{\mu}\right)^2} \tag{22}$$

3 Discrete Event Simulation Program

Now that the theory behind the router performance has been discussed, we can proceed to the core part of the assignment: the experimental verification of the router performance through the implementation of a discrete event simulation program. This program will simulate the random (Poisson) packet arrivals as well as the random (exponential) processing time needed for each packet. As an additional challenge, one requirement of the simulation program is not to rely on existing library functions for random number generation but rather to let the student design its own routines.

In what follows we will first introduce the random number generator. This will be a stand-alone block which will be used by the simulator to draw random numbers from the desired distribution. Then we will discuss the simulator itself, stating any assumption made and justifying design choices. Finally in the next section, we will discuss experimental results.

3.1 Random Number Generator

Even before designing a custom random number generator, the first task is to determine what distribution the random sequence is expected to follow. In our simulator we will be dealing with two random events: the arrival of packets (Poisson distributed) and their servicing time (exponentially distributed). Since,

in a Poisson distribution, the inter-arrival time is exponentially distributed, we actually need only one number generator.

Proof This can be shown easily. Let P_k be the probability of k arrivals in a Poisson random process:

$$P_k = \frac{(\lambda t)^k \cdot e^{-\lambda t}}{k!} \tag{23}$$

Then the probability of no arrivals during a time t is:

$$P_0 = \frac{(\lambda t)^0 \cdot e^{-\lambda t}}{0!} = e^{-\lambda t} \tag{24}$$

Note that from Equation 24, the probability that at least one packet arrives in time t is:

$$P_{k \neq 0} = 1 - e^{-\lambda t} \tag{25}$$

which is the *Cumulative Distribution Function* of an exponential distribution with mean $\frac{1}{\lambda}$:

$$pdf(t) = \begin{cases} \lambda e^{-\lambda t} & \text{, for } t > 0 \\ 0 & \text{, otherwise} \end{cases} \tag{26}$$

Now that it is rather clear that we are after an exponentially distributed random number generator we can start discussing how to actually implement it. One method of obtaining a sequence of exponentially distributed random numbers is to generate a set of uniformly distributed random numbers and then exploit the *Probability Integral Transform* to essentially distort the uniform into an exponential distribution. This is motivated by the fact that a large choice of methods exist for uniform random number generation.

Therefore in the next two paragraphs we are going first to cover the task of generating uniform random numbers and then discuss how the Probability Integral Transform can be used to manipulate uniformly distributed numbers to make them follow any other distribution of interest.

3.1.1 Uniform Random Number Generator

A sequence of uniformly distributed random numbers can be obtained easily with a *Linear Congruential Generator*. The generating function is recursive and obeys the following law:

$$X_{k+1} = a \cdot X_k + c \bmod m \tag{27}$$

Equation 27 yields a pseudo-random sequence with a period P that depends on the choice of a, c and m. In particular the sequence has maximum period only if the two parameters meet strict criteria, namely [2]:

- c and m are relative prime;
- $a - 1$ is divisible by all prime factors of m;
- if m is divisible by 4, so is $a - 1$.

In our implementation of the linear congruential generator, the parameters values chosen are $a = 1664525$, $c = 1013904223$ and $m = 2^{32}$. Therefore Equation 27 becomes:

$$X_{k+1} = 1664525 \cdot X_k + 1013904223 \bmod 2^{32} \tag{28}$$

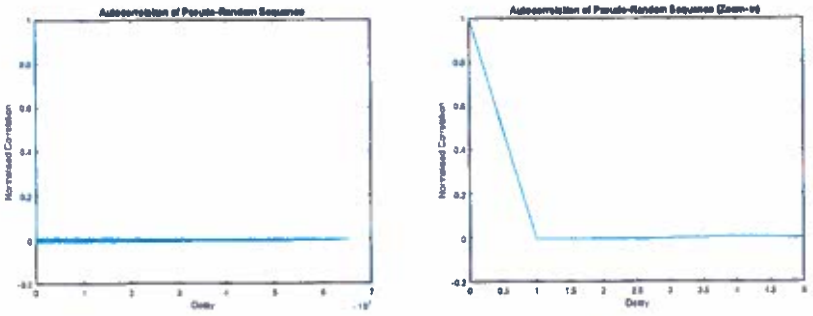
It is easy to see that this combination of a, c and m produces a maximal length sequence of period 2^{32}. Clearly such a period ensures that the sequence does not repeat itself during one simulation run[2].

Linear congruential generators offer a very simple way of producing pseudo-random sequences. They are not state of the art algorithms (e.g. Cryptographically Secure) but they are good enough to be at the core of most library/system functions in programming languages and operating systems.

As a final remark we should note that we are after two random sequences instead of one and ideally we would like the random samples from the two LCGs to be statistically independent. A solution could be to use two subsequences of the same LCG which can be easily achieved by choosing two distinct initial (or seed) values for X_0. Even though a more detailed analysis is outside the scope of this Assignment, we should note that the use of subsequences is motivated by the very good autocorrelation properties of the sequence generated with the chosen LCG.

Figure 1 shows a plot of the autocorrelation over 2^{16} samples.

[2]We will see that the simulation length will be $10s$ with a worst case average of 1900 events per second ($\lambda = 900$, $\mu = 1000$). Obviously $10 \cdot 1900 = 19000 \ll 2^{32}$.



(a) Autocorrelation (b) Zoom-in

Figure 1: Plot of the pseudo-random sequence autocorrelation function.

3.1.2 Exponential Random Number Generator

The transformation of a sequence of uniformly distributed random numbers into a sequence of exponentially distributed random numbers can be achieved through the Probability Integral Transform.

The Probability Integral Transform states that the Cumulative Distribution Function (CDF) $F_X(X)$ of any random variable X is uniformly distributed (i.e. $Y = F_X(X) \sim Unif(0,1)$). We shall prove this fact as follows. The CDF of Y is defined as:

$$F_Y(y) = P(Y \leq y) \tag{29}$$

which by assumption becomes:

$$\begin{aligned} F_Y(y) &= P(F_X(X) \leq y) \\ &= P(X \leq F_X^{-1}(y)) \\ &= F_X(F_X^{-1}(y)) = y \end{aligned} \tag{30}$$

provided that $0 \leq y \leq 1$, otherwise $F_X(x)$ is not invertible. Note that no assumptions have been made on the distribution of X.

We can now use this fact to turn a uniform random experiment into an experiment from any distribution whose CDF is known. In particular for the exponential we have:

$$X_{expo} = -\frac{ln(1 - X_{unif})}{\lambda} \tag{31}$$

where X_{expo} and X_{unif} are respectively the a sample from an exponential and a uniform distribution. This is further shown graphically with the aid of Figure 2.

$1 - X_{unif} \sim Unif(0,$
$If\ X_{unif} \sim Unif(0,$

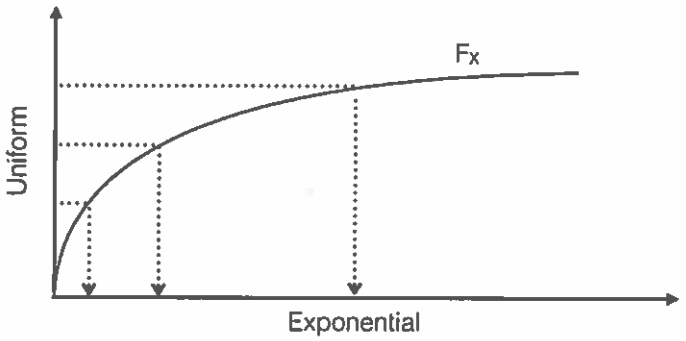


Figure 2: Visualisation of the Probability Integral Transform.

3.2 Implementation

The full simulator code can be found in Appendix A. In this section we will limit ourselves to a brief description of a few design highlights. The analysis has been divided in three parts, each corresponding to a specific function within the program.

3.2.1 Main

The `main` function is first of all responsible for the definition of simulation parameters and the initialisation of the pseudo-random number generator. Once these task are complete, the `main` function also launches several runs of the router simulator and collects statistics at every iteration. Finally, once all the statistics have been collected, the function plots the simulation results as required by the specifications.

The MATLAB code for the `main` function is rather self-explanatory and is listed in Appendix A.1.

3.2.2 Discrete Event Simulator

The `discrete_event_simulator` function is slightly more complicated. The simulator is essentially a state machine with a state `N` which is the instantaneous number of packets in the system. Time in seconds is measured by the variable `wall_clock`.

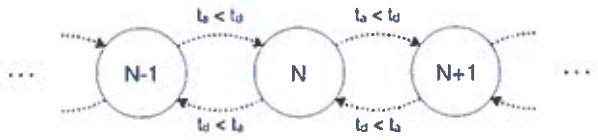


Figure 3: System state diagram for $N > 0$.

The state transitions are conditional to the realisation of two random variables: the next packet arrival time t_a (in the code `input_time`) and the next packet departure time t_d (in the code `service_time`). In particular, if $t_a < t_d$ the clock jumps to t_a and a packet is added to the queue; otherwise the clock jumps to t_d and a packet is *processed* and removed from the queue. Clearly if the queue is empty the processor is disabled and the clock jumps unconditionally to the next packet arrival time.

Every time an event occurs, the time of the next event has to be calculated. From previous sections we know that both the inter-arrival and inter-service time are exponentially distributed with parameters λ and μ respectively. It follows that after a packet arrival event, the next packet arrival time will be calculated as:

$$t_a = t + expo(\lambda) \tag{32}$$

while after a packet departure event, the next packet departure time will be:

$$t_d = t + expo(\mu) \tag{33}$$

provided that the queue is not empty. In both equations t represents the current time (in the code `wall_clock`).

3.2.3 Exponential Random Number Generator

In the code, an exponential pseudo-random number is drawn from the distribution by invoking the `exp_rand(lambda, k)` where the first argument is the parameter of the exponential distribution and the second argument is to select the pseudo-random subsequence which to draw the number from. Every subsequence should be initialised with a distinct seed before selecting it.

When called, the function `exp_rand(lambda, k)` generated an uniformly distributed random number by calling the low level function `LCG(reset, k)` and applies the Probability Integral Transform to it.

3.2.4 Calculation of Statistics

To make the code more efficient both in terms of memory and CPU utilisation, statistics are calculated on-line rather than from a set of collected data at the end of the simulation. The estimate of the mean of any of the parameters is calculated recursively as:

$$\hat{\bar{X}} = \frac{\hat{\bar{X}} \cdot t_{k-1} + X \cdot (t_k - t_{k-1})}{t_k} \tag{34}$$

Similarly the estimate of the variance is calculated as:

$$\hat{\sigma}_X^2 = E[X^2] - \hat{\bar{X}}^2 \tag{35}$$

where $E[X^2]$ is estimated as:

$$\hat{\bar{X}}^2 = \frac{\hat{\bar{X}}^2 \cdot t_{k-1} + X^2 \cdot (t_k - t_{k-1})}{t_k} \tag{36}$$

In practice, since the simulation length is known, both expressions can be further simplified as shown in the code in Appendix A.2.

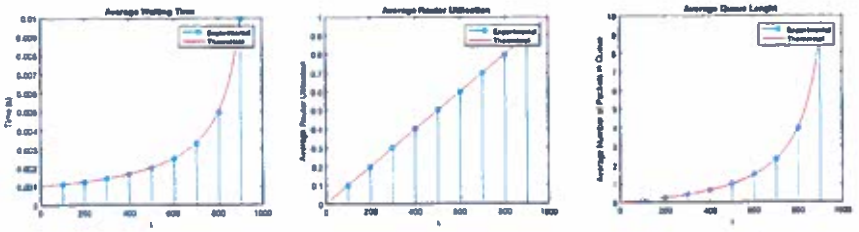
You should provide more details on the measurements recorded from the simulations in order to compute the performance metrics

3.3 Experimental Results

In this section we present and comment on the experimental results obtained from the discrete event simulator. We start with the required observation of the first order statistics: the mean router utilisation, the average queue length and average waiting time of a packet. Out of interest, immediately after we also have a look at the second order statistics to better characterise the steady state behaviour of the router.

3.3.1 First Order Statistics

The first order statistics obtained after 50 simulation runs are shown in Figure 4. Note that the theoretical values are also plotted and marked in orange.



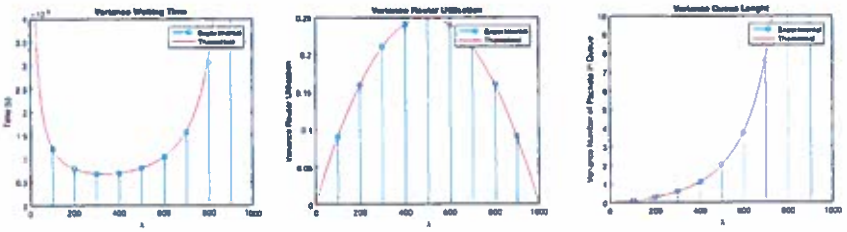
(a) $E[T_w](\lambda)$ (b) $E[U](\lambda)$ (c) $E[N](\lambda)$

Figure 4: Illustration of (a) the mean waiting time, (b) the mean utilisation and (c) average queue length for values of $\lambda \in [100\ 900]$ in steps of 100. Theoretical values shown in orange.

Clearly, the experimental results obtained are perfectly consistent with the theoretical expectations calculated in Section 2.

3.3.2 Second Order Statistics

It is interesting to analyse the second order statistics too. This is because they provide more insight into the stochastic variations that may affect the router utilisation, queue length and packet waiting time. Figure 5 shows plots of the variance of the three quantities. Once again, theoretical curves obtained from Section 2 are shown in orange.



(a) $Var[T_w](\lambda)$ (b) $Var[U](\lambda)$ (c) $Var[N](\lambda)$

Figure 5: Illustration of (a) the variance of the waiting time, (b) the variance of the router utilisation and (c) the variance of the queue length for values of $\lambda \in [100\ 900]$ in steps of 100. Theoretical values shown in orange.

3.3.3 Closing Remarks

Finally, it is rather clear that throughout the analysis we have only considered values of λ such that $\lambda < \mu$. The reason behind this particular choice is that

most of the analysis develops from the convergence of the geometric series:

$$\lim_{N\to\infty}\sum_{n=0}^{N}\left(\frac{\lambda}{\mu}\right)^{n}=\lim_{N\to\infty}\frac{1-\left(\frac{\lambda}{\mu}\right)^{N+1}}{1-\frac{\lambda}{\mu}} \tag{37}$$

Obviously the series does not converge for $\left|\frac{\lambda}{\mu}\right| \geq 1$.

Depending on the assumptions on the queue, one could have two different behaviour of the system. Under infinite queue assumption, a steady state of the system does not exist. The queue grows to infinity and so does the expected value of the waiting time; the router utilisation will be 1 with probability 1.

Under finite queue assumption, the router itself will be stable but will stop accepting packets at a rate of λ. Rather it will accept packets at a rate of μ and reject any input packet if the queue is full. In case of finite queue, first and second order statistics exist, are finite and can be calculated in a closed form.

References

[1] Beranek and Newman. A history of the arpanet: The first decade. 1981.

[2] Dobell A. R. Hull T. E. Random number generators. *Society for Industrial and Applied Mathematics*, 1962.

A Code Listing

A.1 Main Function

```
%% Router Simulator

close all;
clear all;

%% Parameters

sim_time = 10; %seconds

mu = 1/0.001; %Service time


%% Initialisation

%Reset random generator seed
LCG(1, 1);

%Add first subsequence
LCG(2, 1);

%Add second subsequence
LCG(2, 1);

%% Simulation

n_repetitions = 50;

lambdas = 100:100:900;
average_time_in_system = zeros(1, length(lambdas));
average_router_utilisation = zeros(1, length(lambdas));
average_queue_length = zeros(1, length(lambdas));

std_time_in_system = zeros(1, length(lambdas));
std_router_utilisation = zeros(1, length(lambdas));
std_queue_length = zeros(1, length(lambdas));

for i = 1:length(lambdas)

    lambda = lambdas(i);

    for k = 1:n_repetitions

```

```
43          %Optionally reset seed
44          %reset_rnd();
45
46          [input, output, N_mean, T_util, N_vec, time_vec]
                = discrete_event_simulator(sim_time, mu,
                lambda);
47
48          %Optionally plot full simulation results
49          %figure;
50          %stairs(time_vec, N_vec);
51
52          average_time_in_system(i) = (
                average_time_in_system(i)*(k 1) + N_mean / (
                input / sim_time))/k;
53          average_router_utilisation(i) = (
                average_router_utilisation(i)*(k 1) + T_util)/
                k;
54          average_queue_length(i) = (average_queue_length(i
                )*(k 1) + N_mean)/k;
55
56          std_time_in_system(i) = (std_time_in_system(i)*(k
                1) + N_stats.var / (input / sim_time)^2)/k;
57          std_router_utilisation(i) = (
                std_router_utilisation(i)*(k 1) + T_stats.var)
                /k;
58          std_queue_length(i) = (std_queue_length(i)*(k 1)
                + N_stats.var)/k;
59
60
61          [i k]
62
63      end
64
65  end
66
67
68  %% First Order Statistics
69
70  lambdas_theory = 0:1:1000;
71  average_time_in_system_theory = (1/mu) ./ (1 (
        lambdas_theory./mu));
72  router_utilisation_theory = (lambdas_theory) ./ (mu);
73  queue_length_theory = (lambdas_theory./mu) ./ (1 (
        lambdas_theory./mu));
74
75  figure;
```

```
76 stem(lambdas, average_time_in_system);
77 hold on;
78 grid on;
79 plot(lambdas_theory, average_time_in_system_theory);
80 legend('Experimental', 'Theoretical');
81 title('Waiting Time');
82 xlabel('\lambda');
83 ylabel('Time (s)');
84 ylim([0 0.01]);
85 set(gcf,'PaperUnits','inches','PaperPosition',[0 0 5 4])
      %[10 4] for 2 or 3 plots [5 4] for single plot
86 print -dpng ../Report/images/sim_1_a.png -r100
87
88 figure;
89 stem(lambdas, average_router_utilisation);
90 hold on;
91 grid on;
92 plot(lambdas_theory, router_utilisation_theory);
93 legend('Experimental', 'Theoretical');
94 title('Average Router Utilisation');
95 xlabel('\lambda');
96 ylabel('Average Router Utilisation');
97 set(gcf,'PaperUnits','inches','PaperPosition',[0 0 5 4])
      %[10 4] for 2 or 3 plots [5 4] for single plot
98 print -dpng ../Report/images/sim_1_b.png -r100
99
100 figure;
101 stem(lambdas, average_queue_length);
102 hold on;
103 grid on;
104 plot(lambdas_theory, queue_length_theory);
105 legend('Experimental', 'Theoretical');
106 title('Average Queue Lenght');
107 xlabel('\lambda');
108 ylabel('Average Number of Packets in Queue');
109 ylim([0 10]);
110 set(gcf,'PaperUnits','inches','PaperPosition',[0 0 5 4])
      %[10 4] for 2 or 3 plots [5 4] for single plot
111 print -dpng ../Report/images/sim_1_c.png -r100
112
113 %% Second Order Statistics
114
115 lambdas_theory = 0:1:1000;
116 std_time_in_system_theory = (1./(mu.*lambdas_theory)) ./
      (1 (lambdas_theory./mu)).^2;
```

```
117 std_router_utilisation_theory = (lambdas_theory) ./ (mu)
        ((lambdas_theory) ./ (mu)).^2;
118 std_queue_length_theory = (lambdas_theory./mu) ./ (1 (
    lambdas_theory./mu)).^2;
119
120 figure;
121 stem(lambdas, std_time_in_system);
122 hold on;
123 grid on;
124 ylim([0 0.01]);
125 loglog(lambdas_theory, std_time_in_system_theory);
126 legend('Experimental', 'Theoretical');
127 title('Waiting Time');
128 xlabel('\lambda');
129 ylabel('Time (s)');
130 xlim([0 1000]);
131 ylim([0 4e 5]);
132 set(gcf,'PaperUnits','inches','PaperPosition',[0 0 5 4])
      %[10 4] for 2 or 3 plots [5 4] for single plot
133 print  dpng ../Report/images/sim_2_a.png  r100
134
135 figure;
136 stem(lambdas, std_router_utilisation);
137 hold on;
138 grid on;
139 plot(lambdas_theory, std_router_utilisation_theory);
140 legend('Experimental', 'Theoretical');
141 title('Average Router Utilisation');
142 xlabel('\lambda');
143 ylabel('Average Router Utilisation');
144 xlim([0 1000]);
145 set(gcf,'PaperUnits','inches','PaperPosition',[0 0 5 4])
      %[10 4] for 2 or 3 plots [5 4] for single plot
146 print  dpng ../Report/images/sim_2_b.png  r100
147
148 figure;
149 stem(lambdas, std_queue_length);
150 hold on;
151 grid on;
152 ylim([0 10]);
153 loglog(lambdas_theory, std_queue_length_theory);
154 legend('Experimental', 'Theoretical');
155 title('Average Queue Lenght');
156 xlabel('\lambda');
157 ylabel('Average Number of Packets in Queue');
158 xlim([0 1000]);
```

```
ylim([0 10]);
set(gcf,'PaperUnits','inches','PaperPosition',[0 0 5 4])
    %[10 4] for 2 or 3 plots [5 4] for single plot
print  dpng ../Report/images/sim_2_c.png  r100
```

A.2 Simulator

```
function [ input, output, N_mean, T_util, N_vec, time_vec
    ] = discrete_event_simulator( sim_time, mu, lambda )
% Discrete event simulator

%% Init Output Variables
N_vec = [0];
time_vec = [0];
output = 0;
input = 0;
N_mean = 0;
T_util = 0;
N_sq_mean = 0;
T_sq_util = 0;

%% Init State Variables
N = 0;
wall_clock = 0;

%% Simulation
service_time = sim_time;
input_time = exp_rand(lambda, 2);

while input_time < sim_time || service_time < sim_time

    if service_time < input_time %Checks next event

        if N > 0 %Process a packet only if the queue is
            non empty

            N = N    1; %Packet on the output line

            %Update time
            wall_clock = service_time;
            service_time = wall_clock + exp_rand(mu, 1);

            if N == 0 %If queue becomes empty switch off
                processor
                service_time = sim_time;
```

```
            end

            %Stats:
            output = output + 1;
        end
    else

        N = N + 1; %Packet in queue

        %Update time
        wall_clock = input_time;
        input_time = wall_clock + exp_rand(lambda, 2);

        if N == 1 %If queue was empty switch on processor
            service_time = wall_clock + exp_rand(mu, 1);
        end

        %Stats:
        input = input + 1;
    end

    %Stats:
    N_vec = [N_vec N];
    time_vec = [time_vec wall_clock];

    N_mean = N_mean + (N_vec(end-1) * (time_vec(end) -
        time_vec(end-1))) / sim_time;
    T_util = T_util + min(N_vec(end-1), 1) * (time_vec(
        end) - time_vec(end-1)) / sim_time;

    N_sq_mean = N_sq_mean + ((N_vec(end-1))^2 * (time_vec
        (end) - time_vec(end-1))) / sim_time;
    T_sq_util = T_sq_util + min(N_vec(end-1), 1) * (
        time_vec(end) - time_vec(end-1)) / sim_time;

    %Optionally display time evolution (slows down
        execution)
    %[wall_clock N]
end

N_var = N_sq_mean - N_mean^2;
T_var = T_sq_util - T_util^2;

N_stats.mean = N_mean;
N_stats.var = N_var;
```

```
77 T_stats.mean = T_util;
78 T_stats.var = T_var;
79
80 end
```

A.3 Random Number Generator

```
1  function [ out ] = exp_rand( lambda, subsequence_select )
2
3      %Probability Integral Transform
4      out =  log(1   LCG(0, subsequence_select)) / lambda;
5
6  end
```

```
1
2  function [ out ] = LCG( reset, dist_number )
3
4      persistent sequence;
5
6      m = 32; %32 bit random generator
7
8      if reset == 1 %Clear all
9          sequence = [];
10         return
11     elseif reset == 2 %Add a subsequence
12         %Calculate initial seed from the clock
13         sequence = [sequence round(mod(now*24*60*60*1000,
               2^m))];
14
15         %Prevent consecutively added sequences to have
               the same initial
16         %seed
17         pause on
18         pause(0.1);
19         pause off
20
21         return
22     end
23
24     %Step sequence
25     sequence(dist_number) = mod(1664525 * sequence(
           dist_number) + 1013904223, 2^m);
26
27     %Sequence  > Uniform(0, 1)
```

```
28      out = sequence(dist_number) / (2^(m));
29
30  end
```